DO NOW — RECALL

Stacks and queues with arrays

Name: _____ Class: _____ Date: _____

1. What does scores[2] access in a C# array?

2. What is the difference between an array and a List<T> in C#?

3. What does the start pointer identify in a linked list?

4. For data = [10, 20, 30, 40], next = [2, -1, 1, 0] and start = 3, list the data values visited in order by following next pointers until -1.

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

The third element, because array indexing starts at zero.

2. Model answer

An array has a fixed length after creation; a List<T> can grow and shrink.

3. Model answer

The location or index of the first node in the logical sequence.

4. Model answer

40 → 10 → 30 → 20, then -1 ends the traversal.

By the end, you can…

• explain LIFO and FIFO behaviour

• implement a stack using an array and top pointer

• implement a queue using an array with front and rear pointers

Vocabulary — know these terms

Recall the term — stack

Recall the term — queue

Recall the term — LIFO

Recall the term — FIFO

Recall the term — push

Recall the term — pop

Recall the term — enqueue

Recall the term — dequeue

NEW LEARNING

Read, annotate and explain

Same array idea — different pointer rules

STACK
top → newest item

QUEUE
front → oldest item
rear → newest item

This lesson uses arrays and integer pointers, so no OO or library Stack/Queue classes are required.

Stack array

-1 means the stack is empty.

Push and pop

Before push, reject top==stack.Length-1 (full); before pop, reject top==-1 (empty). Push then increments top; pop reads then decrements it.

Queue array

rear tracks the newest item and front tracks the oldest.

Enqueue and dequeue

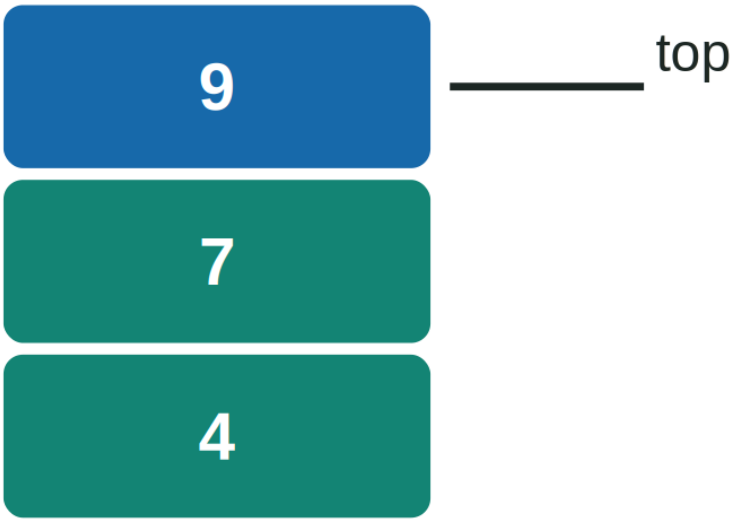
This linear queue uses rear as the last occupied index. Reject enqueue when rear==queue.Length-1 and dequeue when front>rear. Freed earlier slots are not reused; a circular queue later adds wraparound.

Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Same array idea — different pointer rules



Stack containing 4 at bottom, 7 in middle and 9 at top, with top pointer at 9

Same array idea — different pointer rules



Linear queue showing front at value 4 and rear at value 9

I DO — WORKED EXAMPLE

Follow the worked method

Trace a stack

Store 8

```
top = 0
stack[0] = 8
```

Store 7

```
top = 1
stack[1] = 7
```

Remove two values

```
first = 7
second = 8
top = -1
```

Store result

```
top = 0
stack[0] = 15
```

WE DO — GUIDED PRACTICE

Try → Discuss → Improve

1. Trace a stack

OCR uses a calculator-style stack with push/pop operations.

```
Start: empty
push 8 → [8]
push 7 → [?]
A: pop two, add, push → [?]
```

Method / working:

2. Stack vs queue

Describe one difference.

```
Stack = ?
Queue = ?
```

Method / working:

3. Push code

Complete array-based push.

```
top = ?;
stack[top] = value;
```

Method / working:

4. Dequeue code

Complete array-based dequeue.

```
value = queue[?];
front = ?;
```

Method / working:

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: Trace a stack OCR uses a calculator-style stack with push/pop operations. Complete the same state CODE: Start: empty push 8 → [8] push 7 → [?] A: pop two, add, push → [?]

DISCUSS — Compare the sequence of steps or states. Identify the first difference in your methods and justify which step is correct.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

Guided 2: Stack vs queue Describe one difference.

DISCUSS — Compare the sequence of steps or states. Identify the first difference in your methods and justify which step is correct.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

Guided 3: Push code Complete array-based push.

DISCUSS — Compare approaches, walk through one test input, and explain how you would detect or correct a mistake.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

Guided 4: Dequeue code Complete array-based dequeue.

DISCUSS — Compare the sequence of steps or states. Identify the first difference in your methods and justify which step is correct.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Implement push/pop
Create an integer stack array and write code that pushes 5, 8 and 3, then pops and prints one item.

Question 2

Implement enqueue/dequeue
Create an integer queue and enqueue 10, 20, 30. Dequeue and print one item.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

Compare order
Use the same values 2, 4, 6 in a stack and queue and print the first removed value from each.

Question 4

Explain an error
Using the OCR 2023 calculator-stack context, explain what happens if an operation tries to pop more values than are stored.

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

A stack removes the newest stored item first, while a queue removes the oldest stored item first.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

Complete the same state

CODE:

Start: empty

push 8 → [8]

push 7 → [8, 7]

A: pop two, add, push → [15]

IMPROVE: Explain one correction to your own reasoning.

Guided 2 — original CHECK

Complete the comparison

CODE:

Stack = ?

Queue = ?

CODE:

Stack = LIFO

Queue = FIFO

IMPROVE: Explain one correction to your own reasoning.

Guided 3 — original CHECK

Fill the same code

CODE:

top = ?;

stack[top] = value;

CODE:

top = top + 1;

stack[top] = value;

IMPROVE: Explain one correction to your own reasoning.

Guided 4 — original CHECK

Fill the same code

CODE:

```
value = queue[?];
```

```
front = ?;
```

CODE:

```
value = queue[front];
```

```
front = front + 1;
```

IMPROVE: Explain one correction to your own reasoning.